使用方式與目錄

Compiler - Ch1-Ch3 考試 C++ 程式集

2026-10-14｜完整八份 C++17｜只用標準函式庫
HTML 為使用者歷屆線索；其他為課程／作業延伸，非出題機率。
151 項新程式檢查＋sanitizer；Linux 編譯成功，尚未 Linux 實跑。

01 HTML parser 與標籤配對 …… p.3 – 6

02 運算式 AST、後序式與求值 …… p.7 – 9

03 Scanner、註解與 longest match …… p.10 – 12

04 通用 epsilon-NFA → DFA …… p.13 – 15

05 FIRST／FOLLOW／SELECT／LL(1) …… p.16 – 18

06 Stack VM 與最小 interpreter …… p.19 – 20

07 ac 作業完整 compiler …… p.21 – 26

08 固定 HW5(a)/(d) NFA／DFA …… p.27 – 29

考場先確認

只編譯選中的一個 .cpp；每份各有 main，不能一起連結。

先讀案例數 T、EOF／空白行分隔規則、輸出字樣與精度。

judge 不傳命令列參數時，直接改 main 的預設 mode。

HTML：一份可能跨行；引號內 > 不是 tag 結尾；END 要配 stack top。

HTML：本版顯式配對，不含瀏覽器 optional end tag／錯誤修復／entity 解碼。

Scanner：先長運算子，再短；完整 ID 後查 keyword；註解不處理字串內符號。

AST：* / 高於 + -；二元同級左結合；unary neg 與二元 minus 分開。

ac：原作業仍是單字母 ID、+ -、先宣告，目標是 dc 而非數值。

NFA：closure 到固定點；move 後 closure；接受後仍讀完整字串。

LL(1)：epsilon 分支加 FOLLOW；衝突不能偷偷選第一條；此工具為補充。

VM：先 pop 右 b、再左 a，做 a op b；本版 PRINT 會 pop。

程式長行在 PDF 視覺換行，行號只印原行首；提交以原 .cpp 為準。

完整契約與測例請看 Obsidian 的程式集及各題筆記。

01 | HTML parser 與標籤配對 | html_parser.cpp

契約：stdin 全部內容為一份 HTML fragment。預設輸出 VALID／INVALID，兩種判定均 exit 0。--tokens 只分詞；--tree 配對後輸出縮排樹；--text 配對後串接原始 TEXT，不自動加末尾換行。非 check 模式遇錯時 stdout 為空、stderr 診斷、exit 1。

範圍：考試子集：ASCII 名稱轉小寫；boolean、單雙引號／無引號屬性；短 DOCTYPE html；13 個 void elements；script/style/title/textarea 讀到匹配結束標籤。一般元素必須顯式關閉；<custom/> 依本版類 XML 政策關閉。沒有瀏覽器錯誤修復、可省略 end tag、namespace/template 特殊 DOM、完整 script tokenizer、DTD、內容模型或 entity 解碼；註解以第一個 --> 結束；名稱限字母／_／: 開頭，後接字母數字／-／_／:。允許空 fragment／多個根及任意 DOCTYPE 位置。

改法：只檢查 tag 配對時保留 START/END 的 stack。若一行一案，把 main 的整份讀取改為外層 getline，但跨行 HTML 必須仍當一案。tag 數／深度在 parse 的 push/pop 更新，路徑由 nodes.children 走訪。VALID/INVALID 字樣只改 main。

```
Compile: g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror html_parser.cpp
```

輸入：

```
<div id="main"><p title="a>b">Hello <b>world</b></p><br><img src=x><!--ok--></div>
```

預設輸出：

```
VALID
```

完整程式：html_parser.cpp

```cpp
// Exam HTML subset: explicit nesting, attributes, comments, void/raw elements.
// Not a browser's error-recovering WHATWG tree builder. C++17, no dependencies.
#include <iostream>
#include <iterator>
#include <set>
#include <sstream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

bool space(char c) { return c == ' ' || c == '\t' || c == '\n' || c == '\r' || c == '\f'; }
bool letter(char c) { return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z'); }
bool name_char(char c) { return letter(c) || (c >= '0' && c <= '9') || c == '-' || c == '_' || c == ':'; }
std::string lower(std::string s) { for (char& c : s) if (c >= 'A' && c <= 'Z') c = static_cast<char>(c + ('a' - 'A')); return s; }
std::string quoted(const std::string& text) {
    std::string result = "\"";
    for (char c : text) {
        if (c == '\n') result += "\\n";
        else if (c == '\r') result += "\\r";
        else if (c == '\t') result += "\\t";
        else { if (c == '\\' || c == '"') result += '\\'; result += c; }
    }
    return result + '"';
}
const std::set<std::string> void_tags{"area","base","br","col","embed","hr","img","input","link","meta","source","track","wbr"};
const std::set<std::string> raw_tags{"script","style","textarea","title"};
struct Token { std::string kind, value; std::vector<std::pair<std::string, std::string>> attrs; bool closed = false; };

std::vector<Token> tokenize(const std::string& s) {
    std::vector<Token> result;
    std::size_t p = 0;
    std::string raw;
    const auto fail = [&](const std::string& why) { throw std::runtime_error(why + " at byte " + std::to_string(p)); };
    const auto skip_space = [&] { while (p < s.size() && space(s[p])) ++p; };
    const auto name = [&] {
        const auto start = p;
        if (p == s.size() || !(letter(s[p]) || s[p] == '_' || s[p] == ':')) fail("expected name");
        while (p < s.size() && name_char(s[p])) ++p;
```

```
            return lower(s.substr(start, p - start));
        };
        while (p < s.size()) {
            if (!raw.empty()) {
                const auto start = p;
                auto end = s.find("</", p);
                while (end != std::string::npos) {
                    auto after = end + 2 + raw.size();
                    if (lower(s.substr(end + 2, raw.size())) == raw) {
                        while (after < s.size() && space(s[after])) ++after;
                        if (after < s.size() && s[after] == '>') break;
                    }
                    end = s.find("</", end + 2);
                }
                if (end == std::string::npos) fail("unclosed raw-text element " + raw);
                if (end != start) result.push_back({"TEXT", s.substr(start, end - start), {}, false});
                p = end; raw.clear();
            }
            if (s[p] != '<') {
                const auto start = p;
                while (p < s.size() && s[p] != '<') ++p;
                result.push_back({"TEXT", s.substr(start, p - start), {}, false});
                continue;
            }
            if (s.compare(p, 4, "<!--") == 0) {
                const auto end = s.find("-->", p + 4);
                if (end == std::string::npos) fail("unclosed comment");
                result.push_back({"COMMENT", s.substr(p + 4, end - p - 4), {}, false});
                p = end + 3; continue;
            }
            if (lower(s.substr(p, 9)) == "<!doctype") {
                p += 9;
                if (p == s.size() || !space(s[p])) fail("expected whitespace after DOCTYPE");
                skip_space();
                if (name() != "html") fail("only DOCTYPE html is supported");
                skip_space();
                if (p == s.size() || s[p] != '>') fail("invalid DOCTYPE");
                ++p; result.push_back({"DOCTYPE", "html", {}, false}); continue;
            }
            ++p;
            const bool closing = p < s.size() && s[p] == '/';
            if (closing) ++p;
            Token t{closing ? "END" : "START", name(), {}, false};
            if (closing) {
                skip_space();
                if (p == s.size() || s[p] != '>') fail("invalid end tag");
                ++p; result.push_back(std::move(t)); continue;
            }
            std::set<std::string> seen;
            while (true) {
                const auto before = p; skip_space();
                if (p == s.size()) fail("unclosed start tag");
                if (s[p] == '>') { ++p; break; }
                if (s[p] == '/' && p + 1 < s.size() && s[p + 1] == '>') { p += 2; t.closed = true; bre
ak; }
                if (p == before) fail("attributes require separating whitespace");
                const auto key = name();
                if (!seen.insert(key).second) fail("duplicate attribute " + key);
                const auto after_name = p; skip_space();
                std::string value;
                if (p < s.size() && s[p] == '=') {
                    ++p; skip_space();
                    if (p == s.size()) fail("missing attribute value");
```

```
                if (s[p] == '\'' || s[p] == '"') {
                    const char quote = s[p++];
                    const auto start = p, end = s.find(quote, p);
                    if (end == std::string::npos) fail("unclosed attribute quote");
                    value = s.substr(start, end - start); p = end + 1;
                } else {
                    const auto start = p;
                    while (p < s.size() && !space(s[p]) && s[p] != '>') {
                        if (s[p] == '"' || s[p] == '\'' || s[p] == '<' || s[p] == '=' || s[p] == '`') fail("invalid unquoted attribute");
                        ++p;
                    }
                    if (p == start) fail("empty unquoted attribute");
                    value = s.substr(start, p - start); // Slash in an unquoted value belongs to the value.
                }
            } else p = after_name;
            t.attrs.push_back({key, value});
        }
        // Supplement policy: XML-style <custom/> closes custom/normal elements too.
        t.closed = t.closed || void_tags.count(t.value) != 0;
        if (!t.closed && raw_tags.count(t.value)) raw = t.value;
        result.push_back(std::move(t));
    }
    return result;
}
struct Node { Token token; std::vector<int> children; };
std::vector<Node> parse(const std::vector<Token>& tokens) {
    std::vector<Node> nodes{{{"DOCUMENT", "", {}, false}, {}}};
    std::vector<int> stack{0};
    for (const auto& token : tokens) {
        if (token.kind == "END") {
            if (stack.size() == 1 || nodes[stack.back()].token.value != token.value)
                throw std::runtime_error("mismatched end tag " + token.value);
            stack.pop_back();
        } else {
            const int id = static_cast<int>(nodes.size());
            nodes.push_back({token, {}});
            nodes[stack.back()].children.push_back(id);
            if (token.kind == "START" && !token.closed) stack.push_back(id);
        }
    }
    if (stack.size() != 1) throw std::runtime_error("unclosed element " + nodes[stack.back()].token.value);
    return nodes;
}
int main(int argc, char** argv) {
    std::ios::sync_with_stdio(false); std::cin.tie(nullptr);
    const std::string mode = argc == 1 ? "--check" : argv[1];
    bool classify = false;
    try {
        if (argc > 2 || (mode != "--check" && mode != "--tokens" && mode != "--tree" && mode != "--text"))
            throw std::runtime_error("usage: html_parser [--check|--tokens|--tree|--text]");
        const std::string source{std::istreambuf_iterator<char>(std::cin), {}};
        if (std::cin.bad()) throw std::runtime_error("input read failed");
        classify = true;
        if (source.find('\0') != std::string::npos) throw std::runtime_error("invalid input");
        const auto tokens = tokenize(source);
        std::ostringstream output;
        if (mode == "--tokens") {
            for (const auto& t : tokens) {
                output << t.kind << ' ' << quoted(t.value);
```

```
                if (t.kind == "START") output << (t.closed ? " closed" : " open");
                for (const auto& attr : t.attrs) output << ' ' << attr.first << '=' << quoted(attr.second);
                output << '\n';
            }
        } else {
            const auto nodes = parse(tokens);
            if (mode == "--check") output << "VALID\n";
            else if (mode == "--text") {
                for (const auto& n : nodes) if (n.token.kind == "TEXT") output << n.token.value;
            } else {
                // ponytail: iterative arena traversal; no recursive depth limit.
                std::vector<std::pair<int, int>> pending{{0, 0}};
                while (!pending.empty()) {
                    const auto item = pending.back(); pending.pop_back();
                    const auto& n = nodes[item.first];
                    output << std::string(static_cast<std::size_t>(item.second) * 2, ' ') << n.token.kind << ' ' << quoted(n.token.value) << '\n';
                    for (const auto& attr : n.token.attrs) output << std::string(static_cast<std::size_t>(item.second + 1) * 2, ' ') << '@' << attr.first << '=' << quoted(attr.second) << '\n';
                    for (auto it = n.children.rbegin(); it != n.children.rend(); ++it) pending.push_back({*it, item.second + 1});
                }
            }
        }
        std::cout << output.str(); return std::cout ? 0 : 1;
    } catch (const std::exception& e) {
        if (mode == "--check" && classify) { std::cout << "INVALID\n"; return std::cout ? 0 : 1; }
        std::cerr << "error: " << e.what() << '\n'; return 1;
    }
}
```

02 | 運算式 AST、後序式與求值 | expression.cpp

契約：每行一個完整運算式，無案例數。預設每行印空白分隔的 postfix；--ast 每行印一棵 S-expression；--eval 只接受常數，印 15 位有效數字。空白行非法，完全空的輸入表示零案例。任一案例失敗不發出任何暫存輸出，stderr 診斷、exit 1。

範圍：支援 + - * /、括號、一元正負、小數與 e/E。ID 採 [A-Za-z_][A-Za-z_0-9]*，只供 AST/postfix；求值沒有變數環境。沒有 =、%、^、函式、隱含乘法。long double 拒絕零除／非有限值，但非精確有理數，顯示會捨入。

改法：只畫 AST 用 --ast；只求值把 main 預設改 --eval。求值變數要在 evaluate 葉節點加入符號表。新增 ^ 必須另定右結合及相對 unary minus 的優先序，不能只加入運算子字串。

```
Compile: g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror expression.cpp
```

輸入：

```
a+b*c
(2+3)*4
-2*-3
100-30-1
```

預設輸出：

```
a b c * +
2 3 + 4 *
2 neg 3 neg *
100 30 - 1 -
```

完整程式：expression.cpp

```
// One expression per line; shunting yard builds AST, postfix, or evaluates constants.
// Grammar: expr=term{(+|-)term}; term=unary{(*|/)unary}; unary=(+|-)unary|primary.
#include <cmath>
#include <iomanip>
#include <iostream>
#include <locale>
#include <sstream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>
bool digit(char c) { return c >= '0' && c <= '9'; }
bool letter(char c) { return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || c == '_'; }
bool space(char c) { return c == ' ' || c == '\t' || c == '\r' || c == '\n'; }
int priority(char op) { return op == '~' || op == '`' ? 3 : op == '*' || op == '/' ? 2 : 1; }
struct Node { std::string value; int left = -1, right = -1; };
struct Expression {
    std::vector<Node> nodes;
    int root = -1;
    explicit Expression(const std::string& s) {
        std::vector<int> values;
        std::vector<char> ops;
        const auto apply = [&] {
            const char op = ops.back(); ops.pop_back();
            const bool unary = op == '~' || op == '`';
            if (values.size() < (unary ? 1u : 2u)) throw std::runtime_error("missing operand");
            const int right = values.back(); values.pop_back();
            int left = -1;
            if (!unary) { left = values.back(); values.pop_back(); }
            nodes.push_back({op == '~' ? "neg" : op == '`' ? "pos" : std::string(1, op), left, rig
ht});
            values.push_back(static_cast<int>(nodes.size()) - 1);
        };
        bool need_value = true;
        for (std::size_t p = 0; p < s.size();) {
            if (space(s[p])) { ++p; continue; }
            const auto start = p;
            if (digit(s[p]) || (s[p] == '.' && p + 1 < s.size() && digit(s[p + 1])) || letter(s[p]
)) {
                if (!need_value) throw std::runtime_error("missing operator");
                if (letter(s[p])) { ++p; while (p < s.size() && (letter(s[p]) || digit(s[p]))) ++p
```

```
    ; }
40                  else {
41                      while (p < s.size() && digit(s[p])) ++p;
42                      if (p < s.size() && s[p] == '.') { ++p; while (p < s.size() && digit(s[p])) ++
    p; }
43                      if (p < s.size() && (s[p] == 'e' || s[p] == 'E')) {
44                          ++p; if (p < s.size() && (s[p] == '+' || s[p] == '-')) ++p;
45                          if (p == s.size() || !digit(s[p])) throw std::runtime_error("missing expon
    ent digits");
46                          while (p < s.size() && digit(s[p])) ++p;
47                      }
48                  }
49                  nodes.push_back({s.substr(start, p - start), -1, -1});
50                  values.push_back(static_cast<int>(nodes.size()) - 1); need_value = false;
51                  continue;
52              }
53              char op = s[p++];
54              if (op == '(') {
55                  if (!need_value) throw std::runtime_error("missing operator before '('");
56                  ops.push_back(op); continue;
57              }
58              if (op == ')') {
59                  if (need_value) throw std::runtime_error("missing operand before ')'");
60                  while (!ops.empty() && ops.back() != '(') apply();
61                  if (ops.empty()) throw std::runtime_error("unmatched ')'");
62                  ops.pop_back(); continue;
63              }
64              if (std::string("+-*/").find(op) == std::string::npos) throw std::runtime_error("inval
    id expression character");
65              if (need_value) {
66                  if (op != '+' && op != '-') throw std::runtime_error("missing operand");
67                  ops.push_back(op == '-' ? '~' : '`'); continue;
68              }
69              while (!ops.empty() && ops.back() != '(' && priority(ops.back()) >= priority(op)) appl
    y();
70              ops.push_back(op); need_value = true;
71          }
72          if (need_value) throw std::runtime_error("empty/incomplete expression");
73          while (!ops.empty()) { if (ops.back() == '(') throw std::runtime_error("unmatched '('"); a
    pply(); }
74          if (values.size() != 1) throw std::runtime_error("invalid expression");
75          root = values.back();
76      }
77      long double evaluate() const {
78          std::vector<long double> values(nodes.size());
79          for (std::size_t i = 0; i < nodes.size(); ++i) {
80              const auto& n = nodes[i];
81              if (n.right == -1) {
82                  std::istringstream input(n.value); input.imbue(std::locale::classic());
83                  if (!(input >> values[i]) || input.peek() != std::char_traits<char>::eof()) throw
    std::runtime_error("evaluation requires numeric constants");
84              } else if (n.left == -1) values[i] = n.value == "neg" ? -values[n.right] : values[n.ri
    ght];
85              else {
86                  const auto a = values[n.left], b = values[n.right];
87                  if (n.value == "+") values[i] = a + b;
88                  else if (n.value == "-") values[i] = a - b;
89                  else if (n.value == "*") values[i] = a * b;
90                  else { if (b == 0) throw std::runtime_error("division by zero"); values[i] = a / b
    ; }
91              }
92              if (!std::isfinite(values[i])) throw std::runtime_error("non-finite numeric result");
93          }
```

```
            return values[root] == 0 ? 0 : values[root];
        }
        void postfix(std::ostream& out) const {
            // All nodes were appended after their children, in left-to-right order.
            for (std::size_t i = 0; i < nodes.size(); ++i) out << (i ? " " : "") << nodes[i].value;
            out << '\n';
        }
        void ast(std::ostream& out) const {
            std::vector<std::pair<int, char>> pending{{root, 0}};
            while (!pending.empty()) {
                const auto item = pending.back(); pending.pop_back();
                if (item.second) { out << item.second; continue; }
                const auto& n = nodes[item.first];
                if (n.right == -1) { out << n.value; continue; }
                out << '(' << n.value;
                pending.push_back({0, ')'}); pending.push_back({n.right, 0}); pending.push_back({0, '
    '});
                if (n.left != -1) { pending.push_back({n.left, 0}); pending.push_back({0, ' '}); }
            }
            out << '\n';
        }
    };
    int main(int argc, char** argv) {
        std::ios::sync_with_stdio(false); std::cin.tie(nullptr);
        try {
            const std::string mode = argc == 1 ? "--postfix" : argv[1];
            if (argc > 2 || (mode != "--postfix" && mode != "--ast" && mode != "--eval")) throw std::r
    untime_error("usage: expression [--postfix|--ast|--eval]");
            std::ostringstream output; output.imbue(std::locale::classic()); output << std::setprecisi
    on(15);
            std::string line;
            while (std::getline(std::cin, line)) {
                if (!line.empty() && line.back() == '\r') line.pop_back();
                const Expression expr(line);
                if (mode == "--eval") output << expr.evaluate() << '\n';
                else if (mode == "--ast") expr.ast(output);
                else expr.postfix(output);
            }
            if (std::cin.bad()) throw std::runtime_error("input read failed");
            std::cout << output.str(); return std::cout ? 0 : 1;
        } catch (const std::exception& e) { std::cerr << "error: " << e.what() << '\n'; return 1; }
    }
```

03 | Scanner、註解與 longest match | lexer.cpp

契約：stdin 整份來源到 EOF。每行 KIND "lexeme"，std::quoted 轉義引號／反斜線，末行 EOF。KW／ID／NUM／STRING／CHAR／OP／PUNCT；空白與 //、/* */ 不回傳 token。任何錯誤使 stdout 為空、stderr 診斷、exit 1。

範圍：補充 C-like lexer，非完整 C/C++。ID [A-Za-z_][A-Za-z_0-9]* 與教材的底線限制不同。NUM 允許 .5、10.、e/E；沒有 hex/octal 語意、suffix、raw string、preprocessor、行拼接。CHAR 只容一個 byte／簡單兩字元 escape，不解碼 hex／Unicode escape；STRING 保留詞素，不完整驗證 C escape 意義。block comment 不巢狀。

改法：keyword 改 keywords；多字元運算子改 ops 並維持長到短。若題目要求教材 ID，須在 ID 後驗證每個 _ 後至少一個 alnum、首字元為字母。要位置時集中於游標前進更新行欄，CRLF 算一次換行。

```
Compile: g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror lexer.cpp
```

輸入：

```
int sum=12; // skip
float x=1.2e-3; /*skip*/ sum+=2; print("a>b");
```

預設輸出：

```
KW "int"
ID "sum"
OP "="
NUM "12"
PUNCT ";"
KW "float"
ID "x"
OP "="
NUM "1.2e-3"
PUNCT ";"
ID "sum"
OP "+="
NUM "2"
PUNCT ";"
KW "print"
PUNCT "("
STRING "\"a>b\""
PUNCT ")"
PUNCT ";"
EOF
```

完整程式：lexer.cpp

```
// C-like exam lexer: identifiers, decimal/exponent, strings, comments, longest operators.
// Supplement identifier rule: [A-Za-z_][A-Za-z_0-9]* (not Ch3's restricted underscores).
#include <iomanip>
#include <iostream>
#include <iterator>
#include <set>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>
// 採 ASCII 範圍比較，避免 signed char 直接傳給 cctype 造成問題。
bool digit(char c) { return c >= '0' && c <= '9'; }
// 此補充題的 ID 可用底線；不要直接套用到 Ch3 另有底線限制的原題。
bool letter(char c) { return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || c == '_'; }
bool space(char c) { return c == ' ' || c == '\t' || c == '\r' || c == '\n' || c == '\f' || c == '\v'; }
int main() {
    std::ios::sync_with_stdio(false); std::cin.tie(nullptr);
    try {
        // 整份 source 讀到 EOF；不可用 cin >> s，否則空白後的內容會遺失。
        const std::string s{std::istreambuf_iterator<char>(std::cin), {}};
        if (std::cin.bad()) throw std::runtime_error("input read failed");
        // 先讀完整 ID 再查這張表：intx 是 ID，不是 int 接 x。
        const std::set<std::string> keywords{"int","float","double","char","bool","void","if","else","while","for","return","print"};
        // Longest match：三字元運算子放在二字元前；單字元最後判斷。
```

```
        const std::vector<std::string> ops{"<<=",">>=","++","--","==","!=","<=",">=","&&","||","+=","-=","*=","/=","%=","<<",">>","->","&=","|=","^=","::"};
        // 只在完整掃描成功後輸出 token，避免非法尾端造成半份輸出。
        std::ostringstream output;
        for (std::size_t p = 0; p < s.size();) {
            if (space(s[p])) { ++p; continue; }
            // 註解先於 / 運算子；單行註解遇 CR 或 LF 都結束。
            if (s.compare(p, 2, "//") == 0) { while (p < s.size() && s[p] != '\r' && s[p] != '\n') ++p; continue; }
            // 區塊註解採非巢狀規則；找不到 */ 必須報錯。
            if (s.compare(p, 2, "/*") == 0) {
                const auto end = s.find("*/", p + 2);
                if (end == std::string::npos) throw std::runtime_error("unclosed block comment");
                p = end + 2; continue;
            }
            // start 記住 token 起點；結束位置 p 可用來直接切原始 lexeme。
            const auto start = p;
            std::string kind;
            // ID 一次讀到底，再分成 keyword / identifier，避免錯切前綴。
            if (letter(s[p])) {
                ++p; while (p < s.size() && (letter(s[p]) || digit(s[p]))) ++p;
                kind = keywords.count(s.substr(start, p - start)) ? "KW" : "ID";
            // 小數的點只有接數字才可當 number 起點；獨立 . 是 punctuation。
            } else if (digit(s[p]) || (s[p] == '.' && p + 1 < s.size() && digit(s[p + 1]))) {
                kind = "NUM";
                while (p < s.size() && digit(s[p])) ++p;
                if (p < s.size() && s[p] == '.') { ++p; while (p < s.size() && digit(s[p])) ++p; }
                // 指數的正負號屬於 number；一般 -12 的 - 仍是獨立 operator。
                if (p < s.size() && (s[p] == 'e' || s[p] == 'E')) {
                    ++p; if (p < s.size() && (s[p] == '+' || s[p] == '-')) ++p;
                    if (p == s.size() || !digit(s[p])) throw std::runtime_error("missing exponent digits");
                    while (p < s.size() && digit(s[p])) ++p;
                }
            // 進入 literal 後，// 與 /* 不再當作註解；只找未跳脫的配對引號。
            } else if (s[p] == '\'' || s[p] == '"') {
                const char quote = s[p++];
                // token 的類型與 lexeme 分開存；內容不需轉成執行期字串值。
                kind = quote == '"' ? "STRING" : "CHAR";
                int units = 0;
                while (p < s.size() && s[p] != quote) {
                    if (s[p] == '\r' || s[p] == '\n' || s[p] == '\0') throw std::runtime_error("newline/NUL in literal");
                    // 反斜線連同下一個字元一起吃掉；例如 \" 不會提早結束 string。
                    if (s[p] == '\\') {
                        ++p;
                        if (p == s.size() || s[p] == '\r' || s[p] == '\n' || s[p] == '\0') throw std::runtime_error("incomplete escape");
                    }
                    ++p; ++units;
                }
                if (p == s.size()) throw std::runtime_error("unclosed literal");
                ++p;
                // CHAR 只允許一個 byte 或一個簡單 escape；不是完整 C 字元常量規範。
                if (quote == '\'' && units != 1) throw std::runtime_error("character literal needs one character/escape");
            } else {
                // 先試完整多字元 operator，再退到單字元；不要用固定切兩字元的做法。
                for (const auto& op : ops) if (s.compare(p, op.size(), op) == 0) { p += op.size(); kind = "OP"; break; }
                if (kind.empty()) {
                    const char c = s[p++];
                    // 此處才退到單字元 operator，確保 += 不被切成 + 和 =。
```

```
                        if (std::string("+-*/%=<>!&|^~?:").find(c) != std::string::npos) kind = "OP";
                        else if (std::string("(){}[],;.").find(c) != std::string::npos) kind = "PUNCT"
    ;
                        else throw std::runtime_error("invalid byte at " + std::to_string(start));
                    }
                }
                // std::quoted 是輸出包裝；lexeme 本身仍保留原始大小寫與引號。
                output << kind << ' ' << std::quoted(s.substr(start, p - start)) << '\n';
            }
            // 只有題目要求才保留額外 EOF token；考場可刪此行。
            output << "EOF\n";
            std::cout << output.str(); return std::cout ? 0 : 1;
        } catch (const std::exception& e) { std::cerr << "error: " << e.what() << '\n'; return 1; }
    }
```

04 | 通用 epsilon-NFA → DFA | nfa_to_dfa.cpp

契約：n m start k；k 個 final states；alphabet 字串；m 行 from symbol to。states 0..n-1，# 代表 epsilon，不在 alphabet。其後每行完整待辨識字串，空行為 epsilon，預設 ACCEPT/REJECT。--trace 印每一步 NFA 集合；--table 只印完整 DFA，不處理後面案例。

範圍：n≤256、m≤65536、DFA≤4096 subsets，超過明確失敗。子集構造最壞會指數成長。alphabet 為非空、不重複、非空白的單 byte 字元字串，不含 #；純 epsilon 語言可用未使用的 dummy alphabet 字元。沒有 RE→Thompson NFA parser 或最小化。

改法：1 起算 states 可輸入時減 1；多起點改 initial closure 集合；只求辨識可刪 construct，直接保留 closure/move/direct simulation。需要 Unicode 字母表則改 symbol 表示，不能按 UTF-8 byte 當一個字元。

```
Compile: g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror nfa_to_dfa.cpp
```

輸入：

```
6 7 0 1
5
bcd
0 # 1
0 # 4
1 b 2
2 c 3
3 # 1
3 # 4
4 d 5
d
bcd
bcbcd

bc
bd
bcdd
```

預設輸出：

```
ACCEPT
ACCEPT
ACCEPT
REJECT
REJECT
REJECT
REJECT
```

完整程式：nfa_to_dfa.cpp

```
// Generic epsilon-NFA. Header: n m start k; k finals; alphabet; m edges.
// States 0..n-1. Edge symbol '#' means epsilon. Then one complete word per line.
#include <algorithm>
#include <iostream>
#include <map>
#include <set>
#include <stdexcept>
#include <string>
#include <vector>
using States = std::set<int>;
struct NFA {
    int start = 0;
    States finals;
    std::string alphabet;
    std::vector<std::map<char, States>> edges;
    States closure(States states) const {
        std::vector<int> pending(states.begin(), states.end());
        while (!pending.empty()) {
            const int s = pending.back(); pending.pop_back();
            const auto it = edges[s].find('#');
            if (it != edges[s].end()) for (int next : it->second) if (states.insert(next).second) pending.push_back(next);
        }
        return states;
```

```
    }
    States move(const States& states, char c) const {
        States result;
        for (int s : states) {
            const auto it = edges[s].find(c);
            if (it != edges[s].end()) result.insert(it->second.begin(), it->second.end());
        }
        return result;
    }
    bool accepts(const States& states) const { for (int s : states) if (finals.count(s)) return true; return false; }
};
struct DFA { std::vector<States> states; std::vector<std::vector<int>> next; };
DFA construct(const NFA& nfa) {
    DFA dfa; std::map<States, int> ids;
    const auto record = [&](const States& states) {
        const auto it = ids.find(states);
        if (it != ids.end()) return it->second;
        // ponytail: subset construction can be exponential; fail explicitly above this study ceiling.
        if (dfa.states.size() >= 4096) throw std::runtime_error("DFA exceeds 4096 states");
        const int id = static_cast<int>(dfa.states.size());
        ids[states] = id; dfa.states.push_back(states); dfa.next.emplace_back(nfa.alphabet.size(), 0);
        return id;
    };
    record(nfa.closure({nfa.start}));
    for (std::size_t i = 0; i < dfa.states.size(); ++i) for (std::size_t c = 0; c < nfa.alphabet.size(); ++c) {
        const int target = record(nfa.closure(nfa.move(dfa.states[i], nfa.alphabet[c])));
        dfa.next[i][c] = target;
    }
    return dfa;
}
void print_set(const States& states) {
    std::cout << '{'; bool first = true;
    for (int s : states) { if (!first) std::cout << ','; std::cout << s; first = false; }
    std::cout << '}';
}
int main(int argc, char** argv) {
    std::ios::sync_with_stdio(false); std::cin.tie(nullptr);
    try {
        const std::string mode = argc == 1 ? "--check" : argv[1];
        if (argc > 2 || (mode != "--check" && mode != "--table" && mode != "--trace")) throw std::runtime_error("usage: nfa_to_dfa [--check|--table|--trace]");
        NFA nfa; int n, m, k;
        if (!(std::cin >> n >> m >> nfa.start >> k) || n < 1 || n > 256 || m < 0 || m > 65536 || k < 0 || k > n)
            throw std::runtime_error("invalid NFA header (n<=256, m<=65536)");
        const auto valid = [&](int s) { return s >= 0 && s < n; };
        if (!valid(nfa.start)) throw std::runtime_error("invalid start state");
        for (int i = 0, s; i < k; ++i) {
            if (!(std::cin >> s) || !valid(s) || !nfa.finals.insert(s).second) throw std::runtime_error("invalid/duplicate final state");
        }
        if (!(std::cin >> nfa.alphabet) || nfa.alphabet.find('#') != std::string::npos ||
            std::set<char>(nfa.alphabet.begin(), nfa.alphabet.end()).size() != nfa.alphabet.size()) throw std::runtime_error("invalid alphabet");
        nfa.edges.resize(static_cast<std::size_t>(n));
        for (int i = 0; i < m; ++i) {
            int from, to; std::string symbol;
            if (!(std::cin >> from >> symbol >> to) || !valid(from) || !valid(to) || symbol.size() != 1 ||
```

```
                (symbol != "#" && nfa.alphabet.find(symbol[0]) == std::string::npos)) throw std::runtime_error("invalid edge");
            nfa.edges[from][symbol[0]].insert(to);
        }
        std::string line; std::getline(std::cin, line);
        if (line.find_first_not_of(" \t\r") != std::string::npos) throw std::runtime_error("words must start on the next line");
        const auto dfa = construct(nfa);
        if (mode == "--table") {
            std::cout << "DFA start=0 states=" << dfa.states.size() << '\n';
            for (std::size_t i = 0; i < dfa.states.size(); ++i) {
                std::cout << i << ' '; print_set(dfa.states[i]);
                std::cout << (nfa.accepts(dfa.states[i]) ? " accept" : " reject");
                for (std::size_t c = 0; c < nfa.alphabet.size(); ++c) std::cout << ' ' << nfa.alphabet[c] << "->" << dfa.next[i][c];
                std::cout << '\n';
            }
        } else while (std::getline(std::cin, line)) {
            if (!line.empty() && line.back() == '\r') line.pop_back();
            int state = 0; States nstates = nfa.closure({nfa.start}); bool valid_word = true;
            if (mode == "--trace") { print_set(nstates); }
            for (char c : line) {
                const auto index = nfa.alphabet.find(c);
                if (index == std::string::npos) { valid_word = false; if (mode == "--trace") std::cout << " --invalid--> {}"; break; }
                state = dfa.next[state][index]; nstates = nfa.closure(nfa.move(nstates, c));
                if (mode == "--trace") { std::cout << " --" << c << "--> "; print_set(nstates); }
            }
            if (valid_word && nfa.accepts(nstates) != nfa.accepts(dfa.states[state])) throw std::runtime_error("NFA/DFA mismatch");
            if (mode == "--trace") std::cout << ' ';
            std::cout << (valid_word && nfa.accepts(dfa.states[state]) ? "ACCEPT\n" : "REJECT\n");
        }
        if (std::cin.bad()) throw std::runtime_error("input read failed");
        return std::cout ? 0 : 1;
    } catch (const std::exception& e) { std::cerr << "error: " << e.what() << '\n'; return 1; }
}
```

05 | FIRST／FOLLOW／SELECT／LL(1) | ll1.cpp

契約：第一行 P（1..1000），接 P 行 LHS -> RHS，符號空白分隔，一行一個 alternative，# 單獨表示 epsilon。第一 LHS 為 start；所有 LHS 為 nonterminal，其他 RHS 為 terminal。$ 保留 EOF，不寫進 grammar。預設印 FIRST/FOLLOW/SELECT、所有非空 table cells、LL1 YES/NO；--check 讀後續每行 terminal tokens，補 $，印 ACCEPT/REJECT；空行測 epsilon。

範圍：不自動消除左遞迴／left factoring；非 LL(1) 不等於 ambiguous。--check 遇 conflict 報錯；最多 1,000,000 parsing steps。缺少 table cell 表示 error。RHS 名称是否為 nonterminal 由 LHS 集合決定，拼錯名稱會被當 terminal。

改法：只求 FIRST/FOLLOW 可刪 table／recognizer；要求全矩形表格則補印缺格為 error。解析來源 expression 先由 lexer 把實際數字映射為 num token，再送 --check。

```
Compile: g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror ll1.cpp
```

輸入：

```
4
E -> T R
R -> + T R
R -> #
T -> num
```

預設輸出：

```
FIRST(E)={num} FOLLOW(E)={$}
FIRST(R)={#,+} FOLLOW(R)={$}
FIRST(T)={num} FOLLOW(T)={$,+}
SELECT(1)={num}
SELECT(2)={+}
SELECT(3)={$}
SELECT(4)={num}
M[E,num]=1
M[R,$]=3
M[R,+]=2
M[T,num]=4
LL1 YES
```

完整程式：ll1.cpp

```
// FIRST / FOLLOW / SELECT / LL(1) table. P productions, each "A -> X Y".
// One alternative per line; '#' alone is epsilon; '$' is reserved EOF.
// Start symbol = first LHS. --check reads whitespace-separated token sequences afterward.
#include <iostream>
#include <map>
#include <set>
#include <sstream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>
// 符號採字串，所以 num、Expr、ExprTail 不必改成單字元代號。
using Set = std::set<std::string>;
struct Rule { std::string lhs; std::vector<std::string> rhs; };
// 計算整段 RHS 的 FIRST，不是只看第一字元。
// 如果前方可為 epsilon，還要繼續合併下一符號的 FIRST。
Set first_sequence(const std::vector<std::string>& rhs, std::size_t start, const std::map<std::str
ing, Set>& first) {
    Set result;
    for (std::size_t i = start; i < rhs.size(); ++i) {
        // first 表只含 nonterminal；找不到表示 terminal，自己就是 FIRST。
        const auto it = first.find(rhs[i]);
        if (it == first.end()) { result.insert(rhs[i]); return result; }
        for (const auto& t : it->second) if (t != "#") result.insert(t);
        if (!it->second.count("#")) return result;
    }
    // 整段 RHS 都 nullable，或 suffix 是空串，FIRST 才包含 epsilon。
    result.insert("#"); return result;
}
// 只有集合真的增加才回傳 true；用這個旗標決定固定點迴圈能否停止。
bool merge(Set& target, const Set& source) { const auto before = target.size(); target.insert(sour
```

```
    ce.begin(), source.end()); return target.size() != before; }
31  void print_set(const Set& s) { std::cout << '{'; bool first = true; for (const auto& t : s) { if (
    !first) std::cout << ','; std::cout << t; first = false; } std::cout << '}'; }
32  int main(int argc, char** argv) {
33      std::ios::sync_with_stdio(false); std::cin.tie(nullptr);
34      try {
35          const std::string mode = argc == 1 ? "--table" : argv[1];
36          if (argc > 2 || (mode != "--table" && mode != "--check")) throw std::runtime_error("usage:
     ll1 [--table|--check]");
37          int count; std::string line;
38          if (!(std::cin >> count) || count < 1 || count > 1000) throw std::runtime_error("productio
    n count must be 1..1000");
39          std::getline(std::cin, line);
40          if (line.find_first_not_of(" \t\r") != std::string::npos) throw std::runtime_error("produc
    tion starts on next line");
41          std::vector<Rule> rules; Set nonterminals;
42          // 一行一條 production、同一 LHS 的替代規則分開寫；不要直接輸入 |。
43          for (int i = 0; i < count; ++i) {
44              if (!std::getline(std::cin, line)) throw std::runtime_error("missing production");
45              std::istringstream input(line); Rule rule; std::string arrow, symbol;
46              if (!(input >> rule.lhs >> arrow) || arrow != "->" || rule.lhs == "#" || rule.lhs == "
    $" || rule.lhs == "|") throw std::runtime_error("expected A -> symbols");
47              while (input >> symbol) {
48                  if (symbol == "$" || symbol == "|") throw std::runtime_error("reserved symbol; use
     one alternative per line");
49                  rule.rhs.push_back(symbol);
50              }
51              // 不允許省略 epsilon 記號；輸入空 RHS 必須明寫 #。
52              if (rule.rhs.empty()) throw std::runtime_error("write # for epsilon");
53              // # 只可單獨表示 epsilon；內部改用空 vector 表示空 RHS。
54              if (rule.rhs.size() == 1 && rule.rhs[0] == "#") rule.rhs.clear();
55              else for (const auto& s : rule.rhs) if (s == "#") throw std::runtime_error("epsilon mu
    st be alone");
56              nonterminals.insert(rule.lhs); rules.push_back(std::move(rule));
57          }
58          // 先收齊所有 LHS 才知道 nonterminal，避免尚未讀到的規則被當 terminal。
59          std::map<std::string, Set> first, follow;
60          for (const auto& nt : nonterminals) { first[nt] = {}; follow[nt] = {}; }
61          // FIRST 與 FOLLOW 都反覆掃規則到集合不再增加；支援循環相依。
62          bool changed;
63          do {
64              changed = false;
65              for (const auto& rule : rules) if (merge(first[rule.lhs], first_sequence(rule.rhs, 0,
    first))) changed = true;
66          } while (changed);
67          // 第一條 production 的 LHS 是 start；start 的 FOLLOW 包含 EOF $。
68          follow[rules[0].lhs].insert("$");
69          do {
70              changed = false;
71              for (const auto& rule : rules) for (std::size_t i = 0; i < rule.rhs.size(); ++i) {
72                  if (!nonterminals.count(rule.rhs[i])) continue;
73                  // 對 A -> α B β：把 FIRST(β) 去掉 epsilon 後加進 FOLLOW(B)。
74                  auto suffix = first_sequence(rule.rhs, i + 1, first);
75                  const bool nullable = suffix.erase("#") != 0;
76                  if (merge(follow[rule.rhs[i]], suffix)) changed = true;
77                  // β 可空時，再把 FOLLOW(A) 加進 FOLLOW(B)，不能漏掉這個傳播。
78                  if (nullable && merge(follow[rule.rhs[i]], follow[rule.lhs])) changed = true;
79              }
80          } while (changed);
81          // SELECT = FIRST(RHS) 去 epsilon；若 RHS nullable，再加 FOLLOW(LHS)。
82          std::vector<Set> select;
83          std::map<std::pair<std::string, std::string>, std::vector<int>> table;
84          bool ll1 = true;
```

```cpp
        for (std::size_t i = 0; i < rules.size(); ++i) {
            auto s = first_sequence(rules[i].rhs, 0, first);
            if (s.erase("#")) merge(s, follow[rules[i].lhs]);
            select.push_back(s);
            // 表格同一格若有兩條 production 就是 conflict；不可隨便選第一條。
            for (const auto& token : s) { auto& cell = table[{rules[i].lhs, token}]; cell.push_back(static_cast<int>(i)); if (cell.size() > 1) ll1 = false; }
        }
        if (mode == "--table") {
            for (const auto& nt : nonterminals) {
                std::cout << "FIRST(" << nt << ")="; print_set(first[nt]); std::cout << " FOLLOW(" << nt << ")="; print_set(follow[nt]); std::cout << '\n';
            }
            for (std::size_t i = 0; i < rules.size(); ++i) { std::cout << "SELECT(" << i + 1 << ")="; print_set(select[i]); std::cout << '\n'; }
            for (const auto& cell : table) {
                std::cout << "M[" << cell.first.first << ',' << cell.first.second << "]=";
                for (std::size_t i = 0; i < cell.second.size(); ++i) std::cout << (i ? "," : "") << cell.second[i] + 1;
                if (cell.second.size() > 1) std::cout << " CONFLICT";
                std::cout << '\n';
            }
            std::cout << (ll1 ? "LL1 YES\n" : "LL1 NO\n");
        } else {
            // 只能用無衝突的 LL(1) 表做這份 predictive parser。
            if (!ll1) throw std::runtime_error("grammar has LL(1) conflicts");
            while (std::getline(std::cin, line)) {
                std::istringstream input(line); std::vector<std::string> words; std::string word;
                bool accepted = true;
                while (input >> word) { if (word == "$" || word == "#" || nonterminals.count(word)) accepted = false; words.push_back(word); }
                // 輸入自動補 EOF；解析 stack 初始是 $ 與 start，頂端在 vector 尾端。
                words.push_back("$"); std::vector<std::string> stack{"$", rules[0].lhs}; std::size_t pos = 0, steps = 0;
                while (accepted && !stack.empty()) {
                    // ponytail: a finite study budget prevents pathological expansion; raise for larger judges.
                    if (++steps > 1000000) throw std::runtime_error("parse exceeds 1000000 steps");
                    // vector 尾端就是 stack top；push RHS 的順序要反過來。
                    const auto top = stack.back(); stack.pop_back();
                    // terminal 必須匹配 lookahead，才前進輸入位置；nonterminal 只展開，不吃字元。
                    if (!nonterminals.count(top)) { if (top != words[pos]) accepted = false; else ++pos; }
                    else {
                        const auto it = table.find({top, words[pos]});
                        if (it == table.end()) { accepted = false; continue; }
                        // 選到的 RHS 要反向 push，讓最左符號先被 pop／處理。
                        const auto& rhs = rules[it->second[0]].rhs;
                        for (auto r = rhs.rbegin(); r != rhs.rend(); ++r) stack.push_back(*r);
                    }
                }
                // stack 用完且輸入也全用完才接受；只匹配前綴不算成功。
                std::cout << (accepted && pos == words.size() ? "ACCEPT\n" : "REJECT\n");
            }
        }
        if (std::cin.bad()) throw std::runtime_error("input read failed");
        return std::cout ? 0 : 1;
    } catch (const std::exception& e) { std::cerr << "error: " << e.what() << '\n'; return 1; }
}
```

06 | Stack VM 與最小 interpreter | stack_vm.cpp

契約：每行一指令到 EOF：PUSH number、LOAD name、STORE name、ADD/SUB/MUL/DIV/NEG、DUP、PRINT。空行略過。STORE pop 保存；LOAD 複製；PRINT pop 並印 15 位有效數字。EOF 可留剩餘 stack。錯誤時整份 PRINT 暫存不發出，stderr 診斷、exit 1。

範圍：沒有跳躍、條件、函式、地址、型別分析、字串、註解或分號。非任意精度整數／dc。本版 PRINT 會 pop，與 GNU dc 的 p 不 pop 不同。名稱 [A-Za-z_][A-Za-z_0-9]*。

改法：postfix 數字→PUSH，+/-/*//→ADD/SUB/MUL/DIV，neg→NEG，pos→忽略，最後 PRINT。若仿 dc p，把 PRINT 的 pop 改成先檢查 stack 非空再讀 back。

```
Compile: g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror stack_vm.cpp
```

輸入：

```
PUSH 100
STORE x
LOAD x
PUSH 30
PUSH 1
ADD
ADD
STORE x
LOAD x
PRINT
```

預設輸出：

```
131
```

完整程式：stack_vm.cpp

```
// Tiny stack interpreter. One instruction per line; long double arithmetic.
// PUSH number, LOAD name, STORE name, ADD/SUB/MUL/DIV/NEG, DUP, PRINT.
#include <cmath>
#include <iomanip>
#include <iostream>
#include <locale>
#include <map>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>
// 變數名稱首字元是字母或底線，其後才可出現數字。
bool name_valid(const std::string& name) {
    if (name.empty()) return false;
    for (std::size_t i = 0; i < name.size(); ++i) {
        const char c = name[i];
        if (!((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || c == '_' || (i && c >= '0' && c <= '9'))) return false;
    }
    return true;
}
int main() {
    std::ios::sync_with_stdio(false); std::cin.tie(nullptr);
    try {
        // 用 vector.back 當 stack 頂端；沒有固定容量，不必自寫 linked list。
        std::vector<long double> stack;
        // vars 存變數；LOAD 不使用下標默認 0，未定義必須拒絕。
        std::map<std::string, long double> vars;
        // 顯示 15 位有效數字；資料仍保留 long double，不先轉 double。
        std::ostringstream output; output.imbue(std::locale::classic()); output << std::setprecision(15);
        // 共用 pop 先檢查空 stack，避免對空 vector 呼叫 back。
        const auto pop = [&] { if (stack.empty()) throw std::runtime_error("stack underflow"); const auto value = stack.back(); stack.pop_back(); return value; };
        std::string line; int line_number = 0;
        while (std::getline(std::cin, line)) {
            ++line_number;
```

```
            // 每行一條指令：先讀 op，再檢查需要的參數和多餘參數。
            std::istringstream input(line); input.imbue(std::locale::classic());
            std::string op, arg, extra;
            if (!(input >> op)) continue;
            // PUSH/LOAD/STORE 恰好需要一個參數；ADD 等不可多帶 argument。
            const bool needs_arg = op == "PUSH" || op == "LOAD" || op == "STORE";
            if (needs_arg && !(input >> arg)) throw std::runtime_error("missing argument at line " + std::to_string(line_number));
            if (input >> extra) throw std::runtime_error("extra argument at line " + std::to_string(line_number));
            // 數字必須完整解析且有限；1abc 不可偷偷讀成 1。
            if (op == "PUSH") {
                std::istringstream number(arg); number.imbue(std::locale::classic()); long double value;
                if (!(number >> value) || number.peek() != std::char_traits<char>::eof() || !std::isfinite(value)) throw std::runtime_error("invalid finite number");
                stack.push_back(value);
            } else if (op == "LOAD" || op == "STORE") {
                if (!name_valid(arg)) throw std::runtime_error("invalid variable name");
                // STORE 移走頂端存入變數；LOAD 只複製已存在的變數值。
                if (op == "STORE") vars[arg] = pop();
                else { const auto it = vars.find(arg); if (it == vars.end()) throw std::runtime_error("undefined variable " + arg); stack.push_back(it->second); }
            // 本題 PRINT 會 pop；題目若要像 dc 的 p 不 pop，改成讀 back 而不移除。
            } else if (op == "PRINT") { const auto value = pop(); output << (value == 0 ? 0 : value) << '\n'; }
            // NEG 是一元指令，只取一個值；不能與 SUB 的兩個 operand 混用。
            else if (op == "NEG") { const auto value = pop(); stack.push_back(-value); }
            // DUP 把頂端變成兩份；這裡先 pop 再 push 兩次，淨增加一項。
            else if (op == "DUP") { const auto value = pop(); stack.push_back(value); stack.push_back(value); }
            else if (op == "ADD" || op == "SUB" || op == "MUL" || op == "DIV") {
                // postfix 先取右 operand b，再取左 a；8 2 SUB 必須得到 6。
                const auto b = pop(), a = pop(); long double result;
                if (op == "ADD") result = a + b;
                else if (op == "SUB") result = a - b;
                else if (op == "MUL") result = a * b;
                else { if (b == 0) throw std::runtime_error("division by zero"); result = a / b; }
                // 即使 operand 有限，乘法仍可能溢位；結果也要檢查。
                if (!std::isfinite(result)) throw std::runtime_error("non-finite result");
                stack.push_back(result);
            } else throw std::runtime_error("unknown instruction " + op);
        }
        if (std::cin.bad()) throw std::runtime_error("input read failed");
        // 所有 PRINT 暫存到最後；後面有錯誤時不輸出半份執行結果。
        std::cout << output.str(); return std::cout ? 0 : 1;
    } catch (const std::exception& e) { std::cerr << "error: " << e.what() << '\n'; return 1; }
}
```

07 | ac 作業完整 compiler | ac_homework.cpp

契約：stdin 整份 ac；預設 dc 產碼，--tokens／--parse-tree／--ast／--typed-ast／--check；--fold 折疊整數加法。

範圍：原 ac：一個小寫字母 ID、宣告在敘述前、+ -，無括號／乘除／註解；帶 exponent 一律 float。exponent 絕對值≤10000。

改法：模式在 main；token 規格在 scan；int2float 在 check；產碼在 generate。完整說明見原作業筆記。

```
Compile: g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror ac_homework.cpp
```

輸入：

```
i x
x=100
x=x+30+1
p x
```

預設輸出：

```
100
sx
0 k
lx
30
+
1
+
sx
0 k
lx
p
si
```

完整程式：ac_homework.cpp

```
// Ch2 HW1-3 / Ch3 HW3. C++17, stdin -> stdout; errors -> stderr, exit 1.
// One-letter ac identifiers are preserved. Full keywords use longest match.
#include <algorithm>
#include <array>
#include <iostream>
#include <iterator>
#include <sstream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

struct Token { std::string kind, text; std::size_t offset; };
bool digit(char c) { return c >= '0' && c <= '9'; }
bool space(char c) { return c == ' ' || c == '\t' || c == '\r' || c == '\n' || c == '\v' || c == '\f'; }

std::vector<Token> scan(const std::string& source) {
    std::vector<Token> tokens;
    for (std::size_t p = 0; p < source.size();) {
        if (space(source[p])) { ++p; continue; }
        const auto start = p;
        std::string kind;
        const char c = source[p];
        if (digit(c)) {
            kind = "inum";
            while (p < source.size() && digit(source[p])) ++p;
            if (p < source.size() && source[p] == '.') {
                kind = "fnum";
                ++p;
                if (p == source.size() || !digit(source[p]))
                    throw std::runtime_error("lexical: digit required after decimal point at byte " + std::to_string(p));
                while (p < source.size() && digit(source[p])) ++p;
            }
            if (p < source.size() && source[p] == 'e') {
```

```
                    kind = "fnum"; // Supplement: every exponent literal has float type.
                    ++p;
                    if (p < source.size() && (source[p] == '+' || source[p] == '-')) ++p;
                    if (p == source.size() || !digit(source[p]))
                        throw std::runtime_error("lexical: digit required in exponent at byte " + std::to_string(p));
                    while (p < source.size() && digit(source[p])) ++p;
                }
            } else {
                ++p;
                if (c == 'f') { kind = "floatdcl"; if (source.compare(start, 5, "float") == 0) p = start + 5; }
                else if (c == 'i') { kind = "intdcl"; if (source.compare(start, 3, "int") == 0) p = start + 3; }
                else if (c == 'p') kind = "print";
                else if (c >= 'a' && c <= 'z') kind = "id";
                else if (c == '=') kind = "assign";
                else if (c == '+') kind = "plus";
                else if (c == '-') kind = "minus";
                else throw std::runtime_error("lexical: invalid byte at " + std::to_string(start));
            }
            tokens.push_back({kind, source.substr(start, p - start), start});
        }
        tokens.push_back({"$", "", source.size()});
        return tokens;
    }

    // Exact decimal-point movement: never convert a source literal to double.
    std::string decimal(std::string text) {
        constexpr int limit = 10000;
        const auto e = text.find('e');
        int exponent = 0;
        if (e != std::string::npos) {
            auto p = e + 1;
            const bool negative = text[p] == '-';
            if (text[p] == '+' || text[p] == '-') ++p;
            for (; p < text.size(); ++p) {
                if (exponent > (limit - (text[p] - '0')) / 10)
                    throw std::runtime_error("codegen: exponent magnitude exceeds 10000");
                exponent = exponent * 10 + (text[p] - '0');
            }
            if (negative) exponent = -exponent;
            text.resize(e);
        }
        if (text.size() > static_cast<std::size_t>(limit))
            throw std::runtime_error("codegen: mantissa exceeds 10000 characters");
        auto dot = text.find('.');
        int point = static_cast<int>(dot == std::string::npos ? text.size() : dot) + exponent;
        if (dot != std::string::npos) text.erase(dot, 1);
        if (point <= 0) text = "0." + std::string(static_cast<std::size_t>(-point), '0') + text;
        else if (point >= static_cast<int>(text.size())) text.append(static_cast<std::size_t>(point) - text.size(), '0');
        else text.insert(static_cast<std::size_t>(point), ".");
        const auto first = text.find_first_not_of('0');
        if (first == std::string::npos) return "0";
        const auto keep = text[first] == '.' ? first - 1 : first;
        return text.substr(keep);
    }

    enum class Type { none, integer, floating };
    struct Node { std::string label; std::vector<int> children; Type type = Type::none; };
    using Tree = std::vector<Node>; // Arena indices remain valid across vector growth.
    int node(Tree& tree, std::string label, std::vector<int> children = {}) {
```

```
94          tree.push_back({std::move(label), std::move(children), Type::none});
95          return static_cast<int>(tree.size()) - 1;
96      }
97
98      void print_tree(const Tree& tree, int root, std::ostream& out) {
99          std::vector<int> pending{root};
100         while (!pending.empty()) {
101             const int id = pending.back(); pending.pop_back();
102             if (id == -1) { out << ')'; continue; }
103             if (id == -2) { out << ' '; continue; }
104             const auto& n = tree[id];
105             if (n.children.empty()) { out << n.label; continue; }
106             out << '(' << n.label;
107             pending.push_back(-1);
108             for (auto it = n.children.rbegin(); it != n.children.rend(); ++it) {
109                 pending.push_back(*it); pending.push_back(-2);
110             }
111         }
112         out << '\n';
113     }
114
115     struct Parser {
116         const std::vector<Token>& tokens;
117         std::size_t pos = 0;
118         Tree parse, ast;
119         std::vector<int> statements;
120         explicit Parser(const std::vector<Token>& input) : tokens(input) {}
121         bool at(const std::string& kind) const { return tokens[pos].kind == kind; }
122         int take(const std::string& kind) {
123             if (!at(kind)) throw std::runtime_error("syntax: expected " + kind + " at byte " + std::to
    _string(tokens[pos].offset));
124             const auto t = tokens[pos++];
125             if (kind == "$") return node(parse, "$");
126             return node(parse, kind, {node(parse, t.text)});
127         }
128         std::pair<int, int> value() {
129             const auto t = tokens[pos];
130             if (t.kind != "id" && t.kind != "inum" && t.kind != "fnum")
131                 throw std::runtime_error("syntax: expected value at byte " + std::to_string(t.offset))
    ;
132             const int leaf = node(ast, t.kind == "id" ? "id " + t.text : t.text);
133             ast[leaf].type = t.kind == "inum" ? Type::integer : t.kind == "fnum" ? Type::floating : Ty
    pe::none;
134             return {node(parse, "Val", {take(t.kind)}), leaf};
135         }
136         int program() {
137             std::vector<int> declarations, stmts;
138             while (at("intdcl") || at("floatdcl")) {
139                 const auto kind = tokens[pos].kind;
140                 const int keyword = take(kind);
141                 const auto name = tokens[pos].text;
142                 const int id = take("id");
143                 declarations.push_back(node(parse, "Dcl", {keyword, id}));
144                 statements.push_back(node(ast, kind + " " + name));
145             }
146             while (!at("$")) {
147                 if (at("print")) {
148                     const int keyword = take("print");
149                     const auto name = tokens[pos].text;
150                     stmts.push_back(node(parse, "Stmt", {keyword, take("id")}));
151                     statements.push_back(node(ast, "print " + name));
152                     continue;
153                 }
```

```
            const auto name = tokens[pos].text;
            const int target = take("id"), assign = take("assign");
            const auto first = value();
            int expression = first.second;
            std::vector<std::pair<int, int>> tail;
            while (at("plus") || at("minus")) {
                const auto kind = tokens[pos].kind;
                const int op = take(kind);
                const auto next = value();
                tail.push_back({op, next.first});
                expression = node(ast, kind == "plus" ? "+" : "-", {expression, next.second});
            }
            int rest = node(parse, "Expr", {node(parse, "epsilon")});
            for (auto it = tail.rbegin(); it != tail.rend(); ++it) rest = node(parse, "Expr", {it->first, it->second, rest});
            stmts.push_back(node(parse, "Stmt", {target, assign, first.first, rest}));
            statements.push_back(node(ast, "=", {node(ast, "id " + name), expression}));
        }
        int dcls = node(parse, "Dcls", {node(parse, "epsilon")});
        for (auto it = declarations.rbegin(); it != declarations.rend(); ++it) dcls = node(parse, "Dcls", {*it, dcls});
        int seq = node(parse, "Stmts", {node(parse, "epsilon")});
        for (auto it = stmts.rbegin(); it != stmts.rend(); ++it) seq = node(parse, "Stmts", {*it, seq});
        return node(parse, "Prog", {dcls, seq, take("$")});
    }
};

void check(Tree& tree) {
    std::array<Type, 26> symbols{};
    const auto lookup = [&](char c) {
        const auto type = symbols[static_cast<std::size_t>(c - 'a')];
        if (type == Type::none) throw std::runtime_error(std::string("semantic: undeclared identifier ") + c);
        return type;
    };
    const auto promote = [&](int child) {
        if (tree[child].type == Type::floating) return child;
        const int converted = node(tree, "int2float", {child});
        tree[converted].type = Type::floating;
        return converted;
    };
    const auto count = tree.size();
    // Original nodes are in child-before-parent order. New conversion nodes
    // already have a type; never hold a vector reference while adding one.
    for (std::size_t i = 0; i < count; ++i) {
        const auto label = tree[i].label;
        if (label.rfind("intdcl ", 0) == 0 || label.rfind("floatdcl ", 0) == 0) {
            auto& entry = symbols[static_cast<std::size_t>(label.back() - 'a')];
            if (entry != Type::none) throw std::runtime_error("semantic: duplicate declaration " + label);
            entry = label[0] == 'i' ? Type::integer : Type::floating;
            tree[i].type = entry;
        } else if (label.rfind("id ", 0) == 0 || label.rfind("print ", 0) == 0) tree[i].type = lookup(label.back());
        else if (label == "+" || label == "-" || label == "=") {
            const int left = tree[i].children[0], right = tree[i].children[1];
            const auto lt = tree[left].type, rt = tree[right].type;
            if (label == "=" && lt == Type::integer && rt == Type::floating)
                throw std::runtime_error("semantic: cannot assign float to int " + tree[left].label);
            tree[i].type = label == "=" ? lt : (lt == Type::floating || rt == Type::floating ? Type::floating : Type::integer);
```

```
            if (tree[i].type == Type::floating) {
                if (label != "=") { const int c = promote(left); tree[i].children[0] = c; }
                const int c = promote(right); tree[i].children[1] = c;
            }
        }
    }
}

std::string add_integer(std::string a, std::string b) {
    std::string sum;
    int carry = 0;
    while (!a.empty() || !b.empty() || carry) {
        int n = carry;
        if (!a.empty()) { n += a.back() - '0'; a.pop_back(); }
        if (!b.empty()) { n += b.back() - '0'; b.pop_back(); }
        sum.push_back(static_cast<char>('0' + n % 10)); carry = n / 10;
    }
    std::reverse(sum.begin(), sum.end());
    const auto first = sum.find_first_not_of('0');
    return first == std::string::npos ? "0" : sum.substr(first);
}

// Supplement: exact nonnegative integer + folding, matching HW2's x+31.
// Never reassociate subtraction or floating expressions.
void fold(Tree& tree) {
    const auto constant = [&](int id) { return tree[id].type == Type::integer && digit(tree[id].label[0]); };
    const auto count = tree.size();
    for (std::size_t i = 0; i < count; ++i) {
        if (tree[i].label != "+" || tree[i].type != Type::integer) continue;
        const int left = tree[i].children[0], right = tree[i].children[1];
        if (constant(left) && constant(right)) {
            tree[i].label = add_integer(tree[left].label, tree[right].label); tree[i].children.clear();
        } else if (tree[left].label == "+" && constant(tree[left].children[1]) && constant(right)) {
            const int before = tree[left].children[0], c = tree[left].children[1];
            const int sum = node(tree, add_integer(tree[c].label, tree[right].label));
            tree[sum].type = Type::integer;
            tree[i].children = {before, sum};
        }
    }
}

void generate(const Tree& tree, const std::vector<int>& statements, std::ostream& out) {
    for (const int root : statements) {
        std::vector<std::pair<int, bool>> pending{{root, false}};
        while (!pending.empty()) {
            const auto item = pending.back(); pending.pop_back();
            const auto& n = tree[item.first];
            if (!item.second) {
                pending.push_back({item.first, true});
                if (n.label == "=") pending.push_back({n.children[1], false});
                else for (auto it = n.children.rbegin(); it != n.children.rend(); ++it) pending.push_back({*it, false});
                continue;
            }
            if (n.label == "=") out << 's' << tree[n.children[0]].label.back() << "\n0 k\n";
            else if (n.label == "+" || n.label == "-") out << n.label << '\n';
            else if (n.label == "int2float") out << "5 k\n";
            else if (n.label.rfind("id ", 0) == 0) out << 'l' << n.label.back() << '\n';
            else if (n.label.rfind("print ", 0) == 0) out << 'l' << n.label.back() << "\np\nsi\n";
            else if (digit(n.label[0])) out << decimal(n.label) << '\n';
```

```
        }
    }
}

int main(int argc, char** argv) {
    std::ios::sync_with_stdio(false); std::cin.tie(nullptr);
    try {
        std::string mode = "--dc";
        bool optimize = false;
        for (int i = 1; i < argc; ++i) {
            const std::string arg = argv[i];
            if (arg == "--fold") optimize = true;
            else if (arg == "--tokens" || arg == "--parse-tree" || arg == "--ast" || arg == "--typed-ast" || arg == "--check" || arg == "--dc") mode = arg;
            else throw std::runtime_error("usage: ac_homework [--tokens|--parse-tree|--ast|--typed-ast|--check|--dc] [--fold]");
        }
        if (optimize && (mode == "--tokens" || mode == "--parse-tree"))
            throw std::runtime_error("--fold requires --ast, --typed-ast, --check or --dc");
        // ponytail: whole-input scanning; use streaming if source files outgrow RAM.
        const std::string source{std::istreambuf_iterator<char>(std::cin), {}};
        if (std::cin.bad()) throw std::runtime_error("input read failed");
        const auto tokens = scan(source);
        std::ostringstream output; // Publish only after the complete selected phase succeeds.
        if (mode == "--tokens") {
            for (const auto& t : tokens) output << t.kind << (t.text.empty() ? "" : " " + t.text) << '\n';
        } else {
            Parser parser(tokens);
            const int root = parser.program();
            if (mode == "--parse-tree") print_tree(parser.parse, root, output);
            else if (mode == "--ast" && !optimize) print_tree(parser.ast, node(parser.ast, "Prog", parser.statements), output);
            else {
                check(parser.ast);
                if (optimize) fold(parser.ast);
                if (mode == "--check") output << "OK\n";
                else if (mode == "--ast" || mode == "--typed-ast") print_tree(parser.ast, node(parser.ast, "Prog", parser.statements), output);
                else generate(parser.ast, parser.statements, output);
            }
        }
        std::cout << output.str();
        return std::cout ? 0 : 1;
    } catch (const std::exception& error) { std::cerr << "error: " << error.what() << '\n'; return 1; }
}
```

08 | 固定 HW5(a)/(d) NFA／DFA | automata_homework.cpp

契約：5a 辨識 (a|(bc)*d)+；5d 辨識 (bc)*d；一行一案，空行為 epsilon。--table 印原 NFA 邊與完整 DFA。

範圍：只含兩題固定 NFA；32-bit 集合只供 states 1..12；需要任意轉移表時選 nfa_to_dfa。

改法：DOMjudge 不傳參數時，main 固定 5a／5d，再改題目要求的 ACCEPT/REJECT。

```
Compile: g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror automata_homework.cpp
```

下列配套範例使用參數 5a。

輸入：

```
a
d
ad
bcd
bcbcd
abcbcdad
b
bc
abca
bcdd
```

預設輸出：

```
REJECT
ACCEPT
ACCEPT
ACCEPT
ACCEPT
ACCEPT
ACCEPT
REJECT
REJECT
REJECT
ACCEPT
```

完整程式：automata_homework.cpp

```
// Ch3 HW5(d)/(a): original NFA edges -> subset construction -> complete DFA.
#include <algorithm>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

using Set = std::uint32_t;
struct Edge { int from; char symbol; int to; };
struct NFA { std::string alphabet; int start, final; std::vector<Edge> edges; };
struct DFA { std::vector<Set> states; std::vector<std::vector<int>> next; };
Set bit(int state) { return Set{1} << state; }
Set move(const NFA& nfa, Set states, char symbol) {
    Set result = 0;
    for (const auto& e : nfa.edges) if ((states & bit(e.from)) && e.symbol == symbol) result |= bi
t(e.to);
    return result;
}
Set closure(const NFA& nfa, Set states) {
    while (true) {
        const Set next = states | move(nfa, states, '\0');
        if (next == states) return states;
        states = next;
    }
}
DFA construct(const NFA& nfa) {
    DFA dfa;
    const auto record = [&](Set states) {
        const auto it = std::find(dfa.states.begin(), dfa.states.end(), states);
```

```
30          if (it != dfa.states.end()) return static_cast<int>(it - dfa.states.begin());
31          dfa.states.push_back(states);
32          dfa.next.emplace_back(nfa.alphabet.size(), 0);
33          return static_cast<int>(dfa.states.size()) - 1;
34      };
35      record(closure(nfa, bit(nfa.start)));
36      for (std::size_t i = 0; i < dfa.states.size(); ++i) {
37          for (std::size_t c = 0; c < nfa.alphabet.size(); ++c) {
38              const int target = record(closure(nfa, move(nfa, dfa.states[i], nfa.alphabet[c])));
39              dfa.next[i][c] = target; // record may reallocate; assign afterward.
40          }
41      }
42      return dfa;
43  }
44  bool accepts_nfa(const NFA& nfa, const std::string& word) {
45      Set states = closure(nfa, bit(nfa.start));
46      for (char c : word) {
47          if (nfa.alphabet.find(c) == std::string::npos) return false;
48          states = closure(nfa, move(nfa, states, c));
49      }
50      return (states & bit(nfa.final)) != 0;
51  }
52  bool accepts_dfa(const NFA& nfa, const DFA& dfa, const std::string& word) {
53      int state = 0;
54      for (char c : word) {
55          const auto index = nfa.alphabet.find(c);
56          if (index == std::string::npos) return false;
57          state = dfa.next[state][index];
58      }
59      return (dfa.states[state] & bit(nfa.final)) != 0;
60  }
61  void print_set(Set states) {
62      std::cout << '{'; bool first = true;
63      for (int i = 1; i <= 12; ++i) if (states & bit(i)) {
64          if (!first) std::cout << ',';
65          std::cout << i; first = false;
66      }
67      std::cout << '}';
68  }
69  int main(int argc, char** argv) {
70      std::ios::sync_with_stdio(false); std::cin.tie(nullptr);
71      try {
72          if (argc < 2 || argc > 3 || (std::string(argv[1]) != "5d" && std::string(argv[1]) != "5a")
     ||
73              (argc == 3 && std::string(argv[2]) != "--table"))
74              throw std::runtime_error("usage: automata_homework 5d|5a [--table]");
75          // ponytail: fixed homework NFAs fit 32 bits; use a dynamic set for >=32 states.
76          const NFA nfa = std::string(argv[1]) == "5d"
77              ? NFA{"bcd", 1, 6, {{1,0,2},{1,0,5},{2,'b',3},{3,'c',4},{4,0,2},{4,0,5},{5,'d',6}}}
78              : NFA{"abcd", 1, 12, {{1,0,2},{2,0,3},{2,0,6},{3,'a',4},{4,0,5},{5,0,2},{5,0,12},
79                  {6,0,7},{6,0,10},{7,'b',8},{8,'c',9},{9,0,7},{9,0,10},{10,'d',11},{11,0,5}}};
80          const auto dfa = construct(nfa);
81          if (argc == 3) {
82              std::cout << "NFA start=" << nfa.start << " final=" << nfa.final << '\n';
83              for (const auto& edge : nfa.edges) std::cout << edge.from << ' ' << (edge.symbol ? std
    ::string(1, edge.symbol) : "epsilon") << ' ' << edge.to << '\n';
84              std::cout << "DFA start=0 states=" << dfa.states.size() << '\n';
85              for (std::size_t i = 0; i < dfa.states.size(); ++i) {
86                  std::cout << i << ' '; print_set(dfa.states[i]);
87                  std::cout << ((dfa.states[i] & bit(nfa.final)) ? " accept" : " reject");
88                  for (std::size_t c = 0; c < nfa.alphabet.size(); ++c) std::cout << ' ' << nfa.alph
    abet[c] << "->" << dfa.next[i][c];
89                  std::cout << '\n';
```

```
90              }
91          } else {
92              std::string word;
93              while (std::getline(std::cin, word)) {
94                  if (!word.empty() && word.back() == '\r') word.pop_back();
95                  const bool result = accepts_dfa(nfa, dfa, word);
96                  if (result != accepts_nfa(nfa, word)) throw std::runtime_error("NFA/DFA disagreeme
    nt");
97                  std::cout << (result ? "ACCEPT\n" : "REJECT\n");
98              }
99              if (std::cin.bad()) throw std::runtime_error("input read failed");
100         }
101         return std::cout ? 0 : 1;
102     } catch (const std::exception& error) { std::cerr << "error: " << error.what() << '\n'; return
     1; }
103 }
```